# Firmware-Doku PCB_test_code

Diplomarbeit „Analyse von Degradationseffekten bei Feldeffekttransistoren“ · Test-PCB mit STM32F103RB, ADS8681 (ADC), AD5689R (DAC) · Stand 22.10.2026

## 1 Überblick

Der PC schickt Textbefehle über UART (9600 Baud, Zeilenende \r oder \n). Der Mikrocontroller speichert die Werte in der Struct `Measure_Param` und startet je nach Flag eine Messung, einen Stress oder eine Wartezeit. Messdaten gehen über UART zurück.

```
PC  --UART-->  UART.c (Interrupt)  -->  COMMAND_DECODE.c  -->  Measure_Param
                                                                    |
PC  <--UART--  measure.c (IDVG / transient / stress)  <--  main.c Hauptschleife
                        |
          ads8681.c (Strom messen)   AD5689.c (Spannung ausgeben)   relai.c (Bereich)
```

## 2 Dateien

| Datei | Aufgabe |
|---|---|
| `main.c` | Initialisierung, Standardwerte, Hauptschleife |
| `PINs.h / PINs.c` | Pin-Namen (Bit-Banding) und GPIO-Konfiguration |
| `clock.c` | Takt-Einstellungen (APB-Teiler, PLL aus → interner 8-MHz-Takt) |
| `NVIC.c` | Interrupt-Priorität setzen und Interrupt freigeben |
| `UART.c` | Senden, Empfangen per Interrupt, ABORT-Erkennung |
| `SPI.c` | SPI1 als Master, 8-Bit-Übertragung (ADC und DAC teilen sich SPI1) |
| `dma.c` | DMA für SPI1 (RX Kanal 2, TX Kanal 3) |
| `TIMER.c` | TIM2 für Abtastrate, `wait_ms`, `delay_ms_abortable` |
| `ads8681.c` | ADC: Frames, Befehle, Umrechnung Rohwert → Spannung → Strom |
| `AD5689.c` | DAC: Befehle senden, Spannung 0…2,5 V setzen |
| `relai.c` | Bistabile Relais: Messbereich 10 MΩ / 100 kΩ, Eingang auf GND |
| `SCPI_COMMANDS.h` | Liste aller Befehle (Defines) und Enums |
| `COMMAND_DECODE.c/.h` | Struct `Measure_Param` und Befehl → Feld |
| `measure.c/.h` | Messabläufe (IDVG, transient, stress, wait_temp_equi) – noch leer |

## 3 Ablauf in main.c

- Pins, UART und SPI initialisieren, Versorgungen (3,3 V, 5 V analog, OPV) einschalten.
- Relais-Spulen aus, Chip-Selects high, Baudrate 9600, SPI-Takt /64.
- Sicherer Start: Bereich 100 kΩ, TIA-Eingang auf GND, beide DAC-Kanäle 0 V. Danach „SAFDAT READY“.
- Alle Parameter auf Standardwerte (0 / false, Mittelung 100).
- Hauptschleife: Befehl empfangen → decodieren. Danach wird je nach Flag `IDVG()`, `transient()`, `stress()` oder `wait_temp_equi()` aufgerufen.

## 4 Treiber kurz erklärt

**UART.c** – Der Interrupt sammelt Zeichen bis zum Zeilenende, dann ist der String fertig (`uart_string_received()`, `uart_get_string()`). Unabhängig davon prüft er jedes Zeichen auf „ABOR“ und setzt dann `abort_requested = true` – das funktioniert auch während einer Messung.

**SPI.c** – `spi1_transfer8()` sendet und empfängt ein Byte, `spi1_transfer16()` zwei Bytes. Mode: CPOL = 0, CPHA = 1.

**ads8681.c** – Ein Frame hat 32 Bit. `ADS8681_send_Command_blocking()` schreibt einen Befehl und liest danach mit einem NOP-Frame die Antwort. `ADS8681_get_voltage()` rechnet den Rohwert (Straight Binary, 0x8000 = 0 V) in Volt um, `ADS8681_get_current()` in µA:

```
U_ADC = (raw - 32768) * 2 * 3 * VREF / 65536        (Bereich ±3·VREF = ±12,288 V)
I     = U_ADC * 3 / Rf     -> in µA                (Rf = 10 MΩ oder 100 kΩ)
```

**AD5689.c** – Ein Frame hat 24 Bit: 4 Bit Befehl, 4 Bit Adresse (DAC A, B oder beide), 16 Bit Daten. `AD5689_set_Voltage(V, Kanal)` begrenzt auf 0…2,5 V und rechnet in den 16-Bit-Wert um.

**relai.c** – Bistabile Relais bekommen einen 100-ms-Puls auf die Set- oder Reset-Spule. `set_range_10meg()` (100-nA-Bereich) setzt `range_100n = true`, `set_range_100k()` setzt false. `set_GND_Relai()` / `reset_GND_Relai()` legen den TIA-Eingang auf GND bzw. wieder ans DUT.

**TIMER.c** – `wait_ms(ms)` ist eine Warteschleife (auf 8 MHz abgestimmt). `delay_ms_abortable(ms)` wartet ebenso, bricht aber bei ABORT ab und gibt dann true zurück. `init_tim2(f)` stellt TIM2 auf Frequenz f, der Interrupt startet einen ADC-Frame per DMA.

# 5 Befehle

Format: `NAME= wert` (mit Leerzeichen nach dem =), abgeschlossen mit \n. Bei SOURce steht 1 = Gate, 2 = Drain (Annahme, am Schaltplan prüfen).

| Befehl | Wert | Bedeutung |
|---|---|---|
| `SOURce1_VOLTage= / SOURce2_…` | Zahl [V] | Fixe Spannung |
| `SOURce<n>_Sweep_Start= / _Stop= / _Step=` | Zahl [V] | Sweep-Bereich und Schrittweite |
| `SOURce<n>_Sweep_TIme=` | Zahl | Sweep-Zeit |
| `SOURce<n>_PROT=` | Zahl [V] | Spannungsgrenze |
| `SOURce<n>_RANGE=` | 25 / 50 | Ausgangsbereich |
| `SOURce<n>_OUTput= / _AUTOrange=` | ON / OFF | Ausgang / Autorange |
| `SENSE_RANGE=` | 0 / 1 | 0 = 100 nA (10 MΩ), 1 = 10 µA (100 kΩ) |
| `SENSE_AUTORANGE=` | ON / OFF | Autorange Strommessung |
| `SENSE_MODE_GND= / SENSE_MODE_AZERO=` | ON / OFF | Eingang auf GND / Auto-Zero |
| `SENSE_MODE_AVERAGE=` | Zahl | Anzahl Mittelungen |
| `SENSE_MODE_MEASUREMENT=` | START / STOP | Messung starten / stoppen |
| `SENSE_MODE_IDVG=` | TRUE / FALSE | Messart ID-VG |
| `SENSE_MODE_TRANSIENT=` | TRUE / FALSE | Messart Transient |
| `STRESS_TYPE=` | BTI / HCD | BTI: nur Gate · HCD: Gate + Drain |
| `STRESS_MODE=` | STATIC / DYNAMIC | Dauerstress / gepulst |
| `STRESS_VOLTage=` | Zahl [V] | Gate-Stressspannung |
| `STRESS_DRAIN_VOLTage=` | Zahl [V] | Drain-Stressspannung (nur HCD) |
| `STRESS_ON_TIME= / STRESS_OFF_TIME=` | Zahl [ms] | Puls an / aus (DYNAMIC) |
| `STRESS_TIME=` | Zahl [ms] | Gesamtdauer (STATIC) |
| `STRESS_CYCLES=` | Zahl | Anzahl Pulse (DYNAMIC) |
| `STRESS_OUTput=` | ON / OFF | Stress ausführen |
| `STRESS_AFTER_IDVG= / STRESS_AFTER_TRANSIENT=` | TRUE / FALSE | Nach dem Stress messen |
| `Temp_equilibrium_wait=` | TRUE / FALSE | Auf thermisches Gleichgewicht warten |
| `ABORT (oder ABOR)` | – | Sofortiger Abbruch, wird im Interrupt erkannt |

Beispiel ID-VG:

```
SENSE_RANGE= 0
SOURce2_VOLTage= 0.1
SOURce1_Sweep_Start= 0
SOURce1_Sweep_Stop= 2.5
SOURce1_Sweep_Step= 0.05
SENSE_MODE_AVERAGE= 16
SENSE_MODE_IDVG= TRUE
```

# 6 Offene Punkte

**Achtung – UART-Empfang:** `uart1_Rx_Interupt(true)` wird in `main()` nirgends aufgerufen und `set_up_uart1()` setzt RXNEIE nicht. Ohne diese Zeile kommt kein Befehl an (auch kein ABORT).

- `ADS8681_init()` und `AD5689_init()` werden in `main()` nicht aufgerufen (Reset der Chips, DAC-Power-up).
- `measure.c`: `IDVG`, `transient`, `stress`, `wait_temp_equi` sind leer und müssen geschrieben werden.
- In der Hauptschleife werden die Flags nach der Messung nicht zurückgesetzt → die Funktion läuft sonst endlos wieder (am Ende der Funktion auf false setzen).
- ADC-Bereich: RANGE_SEL ist auskommentiert → ±3·VREF. `ADC_FSR_HALF` muss zum eingestellten Bereich passen; ±0,625·VREF wäre für ±2,5 V genauer.
- Takt: Der Chip läuft auf dem internen 8-MHz-Takt (HSI). `wait_ms` und Baudrate hängen davon ab.
- 9600 Baud ist für schnelle Transienten (RTN) zu langsam.
- DAC-Treiberstufe (Verstärkung, Invertierung) ist in `AD5689_set_Voltage` noch nicht eingerechnet.

# 7 Quellen


- TI: ADS8681 Datenblatt – https://www.ti.com/lit/ds/symlink/ads8681.pdf (lokal Hardware/Datasheet/ads8681.pdf)
- Analog Devices: AD5689R Datenblatt – https://www.analog.com/media/en/technical-documentation/data-sheets/AD5689R_5687R.pdf
- ST: RM0008 Reference Manual STM32F10x (GPIO, Bit-Banding, SPI, DMA, TIM, USART) – https://www.st.com/resource/en/reference_manual/rm0008-stm32f101xx-stm32f102xx-stm32f103xx-stm32f105xx-and-stm32f107xx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
- SCPI-99 Standard (Befehlsaufbau, ABORt) – https://www.ivifoundation.org/downloads/SCPI/scpi-99.pdf
- Eigene Doku: Dokumentation/Messroutinen_IDVG_Transient.pdf, Dokumentation/Stressing.md